

Docker

Часть 2

Евгений Мисяков
Teamlead Devops Умскул



Проверка связи



Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти



Поставьте в чат:

-  если меня видно и слышно
-  если нет

Евгений Мисяков

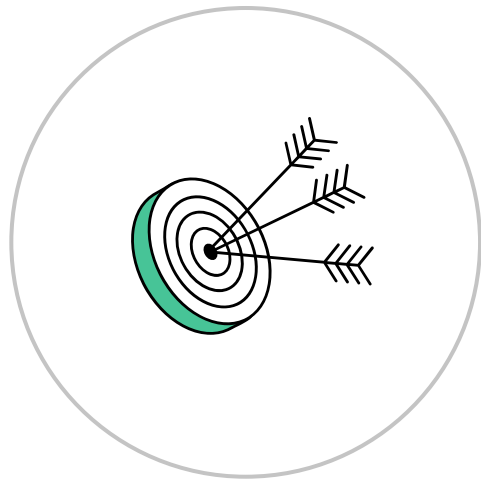
О спикере:

- SRE инженер в Нетологии



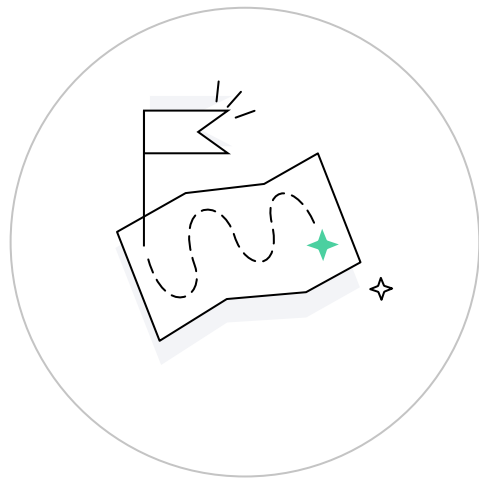
Цели занятия

- Разобраться, что такое Docker Compose и для чего он нужен
- Поговорить о создании и наполнении `docker-compose.yml`
- Запустить одноконтейнерное приложение с помощью Docker
- Запустить многоконтейнерное приложение с помощью Docker Compose
- Поработать с такими сущностями Docker, как `volume` и `network`



План занятия

- 1 Введение в Docker Compose
- 2 Развертывание системы мониторинга с помощью Docker
- 3 Развертывание системы мониторинга с помощью Docker Compose
- 4 Итоги
- 5 Домашнее задание



*Нажмите на нужный раздел для перехода

Вспоминаем прошрое занятие

Вопрос: в чем заключаются преимущества использования контейнеризации?



Вспоминаем прошрое занятие

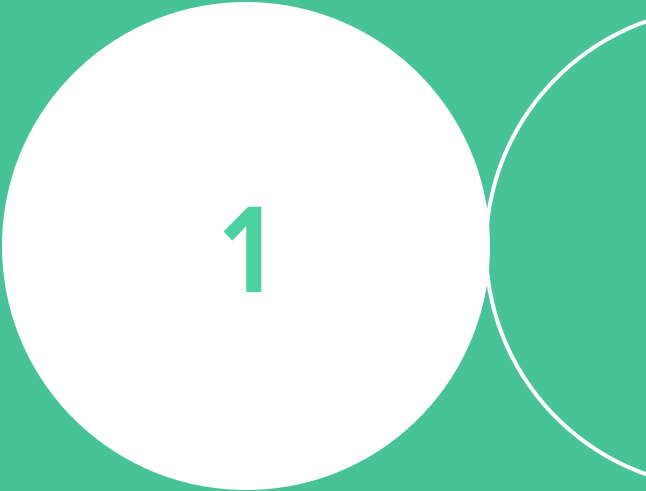
Вопрос: в чем заключаются преимущества использования контейнеризации?

Ответ:

1. Портативность
2. Экономичность
3. Безопасность
4. Версионность



Введение в Docker Compose



1

Docker как основа гибкой инфраструктуры

Мы научились запускать легковесные контейнеры по одиночке, но что будет, когда нам придется запускать множество контейнеров ежедневно? Ведь это норма современных реалий в условиях микросервисной эпохи.

Можно запускать `docker run` столько раз, сколько понадобится. Контейнеры по умолчанию изолированы от хоста, а значит и друг от друга, как тогда будут микросервисы взаимодействовать между собой?

Одного докера не хватит для обеспечения работы ПО, состоящего из нескольких контейнеров. Тут нам на помощь приходит **docker-compose**



Docker Compose — это инструмент, который позволяет управлять многоконтейнерными приложениями с помощью файла конфигурации YAML.

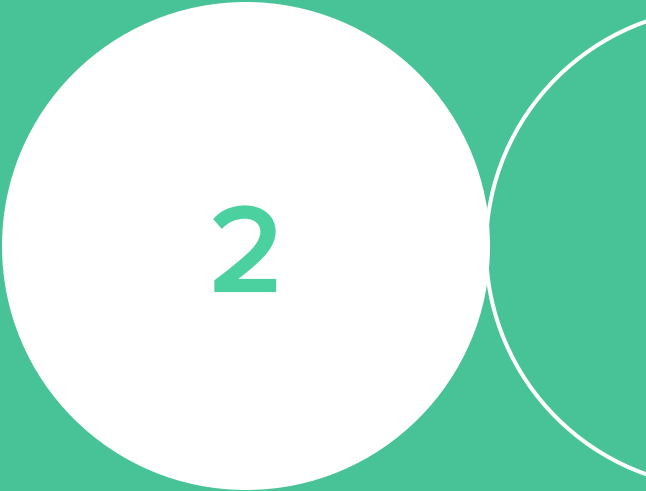
Docker Compose упрощает запуск и оркестрацию контейнеров в различных средах.

Разница между Docker и Docker Compose

Docker применяется для управления отдельными контейнерами (сервисами), из которых состоит приложение.

Docker Compose используется для одновременного управления несколькими контейнерами, входящими в состав приложения

Развертывание системы мониторинга с помощью Docker



2

Prometheus

Prometheus — бесплатное программное приложение, используемое для мониторинга и оповещения о событиях.

Prometheus записывает метрики в базу данных временных рядов, созданную с использованием модели запроса HTTP, гибкими запросами и оповещениями в реальном времени

Деплой Prometheus с помощью Docker

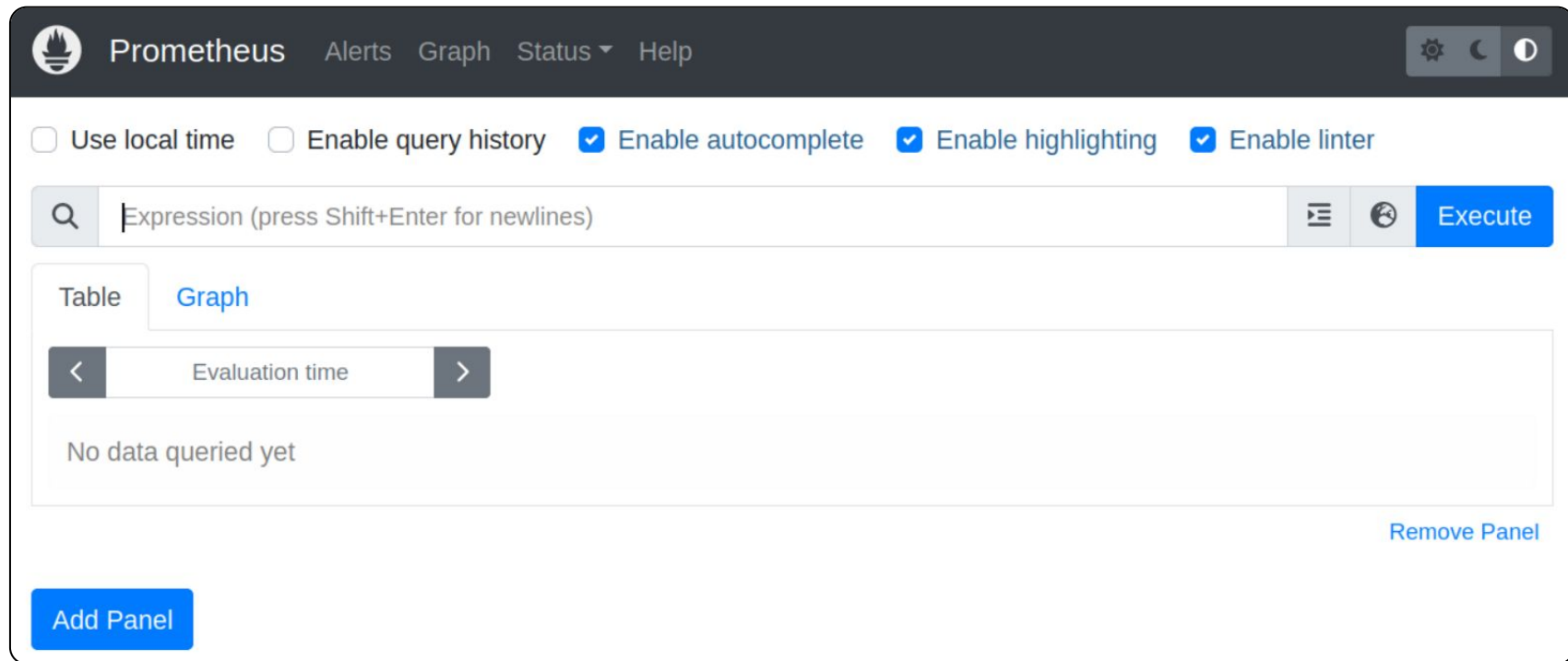
Открываем браузер, переходим в Docker Hub, вбиваем в поиск Prometheus и попадаем на страницу, на которой описывается, как запустить контейнер. Для этого достаточно использовать команду:

```
docker run \  
  -p 9090:9090 \  
  -v ./prometheus/config:/etc/prometheus \  
  --name prometheus \  
  prom/prometheus
```

- **-p** означает, какой порт хоста на какой порт контейнера мы пробрасываем
- **-v** означает, какие тома мы пробрасываем в с хоста внутрь контейнера

Первичная проверка работоспособности

Подключаемся на IP-адрес Docker сервера на внешний порт (9090), нам должна открыться стартовая страница:



Получаем метрики с хоста с помощью node_exporter

Открываем конфигурацию Prometheus и добавляем (если еще нет) эндпоинт для получения метрик из предустановленного node_exporter:

```
- job_name: "docker_server"

# metrics_path defaults to '/metrics'
# scheme defaults to 'http'.

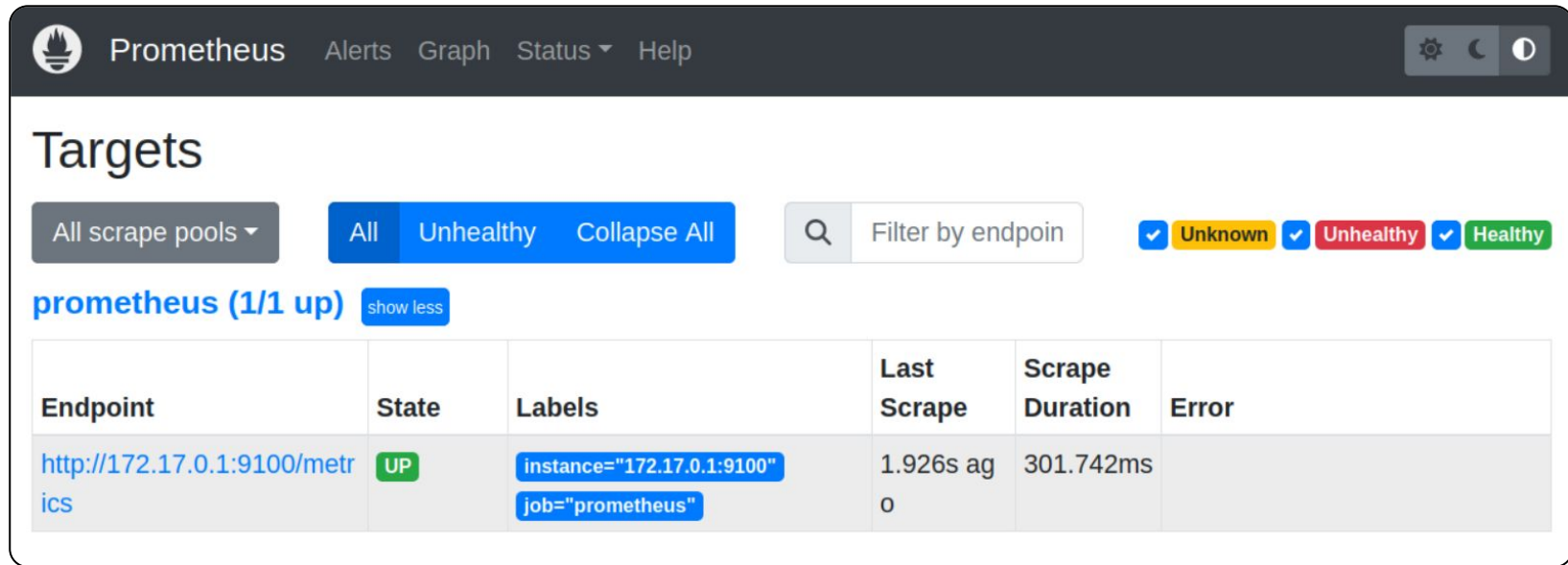
static_configs:
  - targets: ["172.17.0.1:9100"]
```

Перезапускаем контейнер:

```
docker stop prometheus
docker start prometheus
```


Проверяем статус таргета в Prometheus

Видим, что Prometheus начал получать метрики (UP) с докер хоста:



The screenshot shows the Prometheus web interface. The top navigation bar includes the Prometheus logo, the name 'Prometheus', and links for 'Alerts', 'Graph', 'Status', and 'Help'. On the right, there are icons for settings, a moon (dark mode), and a sun (light mode). The main section is titled 'Targets'. Below the title, there are filters: 'All scrape pools' (dropdown), 'All' (selected), 'Unhealthy', and 'Collapse All'. A search bar with the placeholder 'Filter by endpoint' is also present. To the right of the search bar are three status filters: 'Unknown' (yellow), 'Unhealthy' (red), and 'Healthy' (green), each with a checkmark icon. Below the filters, the text 'prometheus (1/1 up)' is displayed, followed by a 'show less' button. A table below shows the target details:

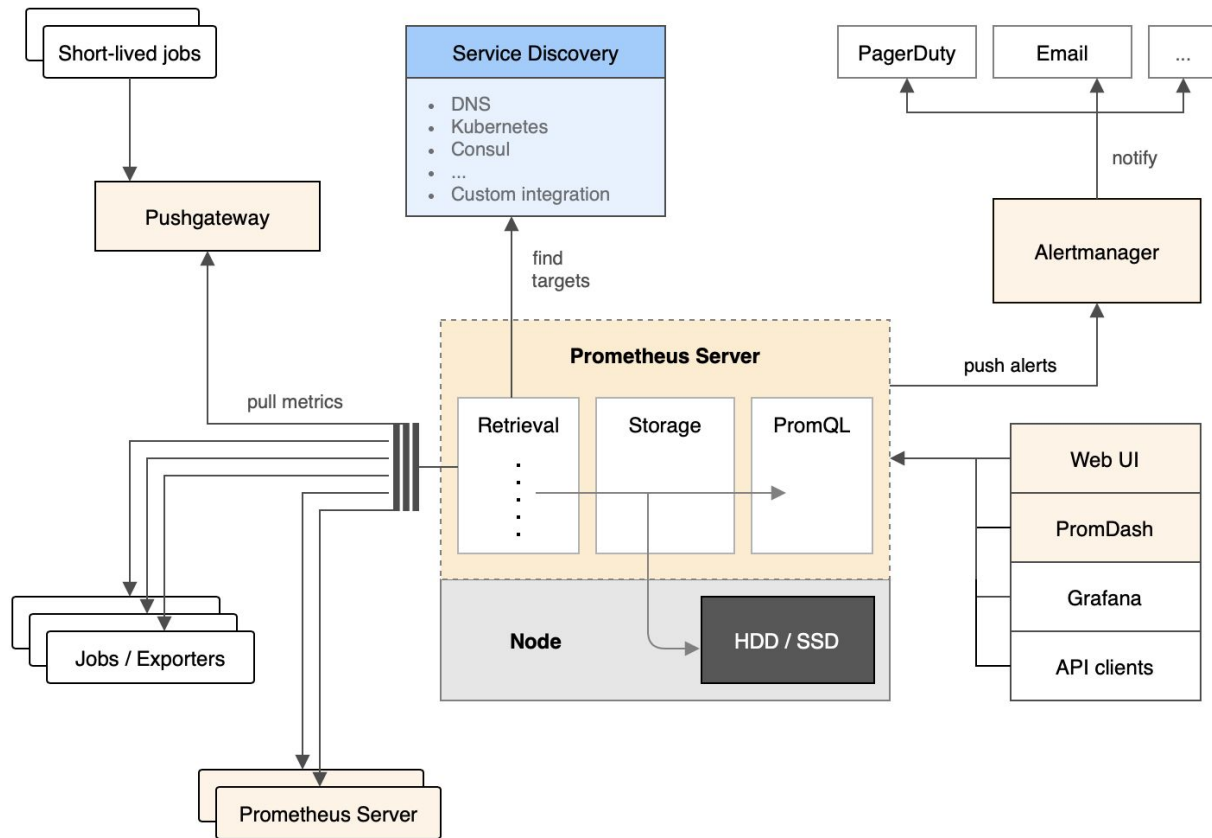
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://172.17.0.1:9100/metrics	UP	<code>instance="172.17.0.1:9100"</code> <code>job="prometheus"</code>	1.926s ago	301.742ms	

172.17.0.1 — шлюз дефолтной подсети докера (можно переопределить в конфигурации)

**А теперь давайте
серьезно**



Архитектура системы мониторинга Prometheus



Что делать, если контейнеров больше одного

Ситуация, когда вы вводите одну команду, запускаете один контейнер, и ваша проблема решается — недостижимый идеал. На практике контейнеров зачастую больше одного.

Продакшн-среда может состоять из десятков серверов с уникальной нагрузкой, и на каждом сервере бегают десятки контейнеров, каждый из которых со своей нагрузкой

Docker Compose

В ситуации с многоконтейнерными решениями на помощь приходит **Docker Compose**.

docker-compose.yml — это единая конфигурация, в которой прописаны образы и то, как они должны быть запущены.

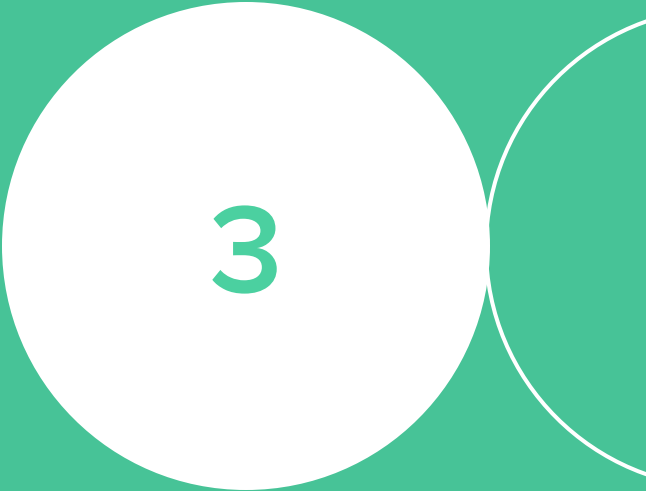
Docker Compose считывает и последовательно исполняет все что указано в **docker-compose.yml**

Prometheus

Prometheus поставляется в виде одного контейнера и может работать независимо, но пользы от него в таком виде будет не очень много. В качестве примера мы будем рассматривать довольно популярный стек:

- Prometheus
- Pushgateway — сервис, в который можно скидывать метрики для последующей обработки Prometheus'ом, когда стандартная pull-модель не применима
- Alertmanager — средство обработки, группировки и отправки оповещений на email или мессенджеры
- Grafana — средство визуализации метрик во времени

Развертывание системы мониторинга с помощью Docker Compose



3

Установка Docker Compose

Устанавливается вместе с Docker Engine (см. предыдущий урок), пакет имеет название **docker-compose-plugin**.

За самым актуальным способом установки Docker в нужную систему лучше обращаться к [официальной документации](#)

Структура docker-compose.yml

Опишем основу нашего сценария docker-compose.yml

```
version: '3'
services:
  netology-app

volumes:
  netology-lesson:

networks:
  netology-lesson:
```

- **version** — это поддерживаемая версия движка
- **services** — список запускаемых контейнеров
- **volumes** определяет, какие данные необходимо сохранить на хостовой файловой системе
- **network** задаёт статическую подсеть, в которой будут запущены контейнеры



Важные замечания к выполнению домашнего задания:

- **отступы важны, так как определяют структуры данных**
- **контейнеры обращаются друг к другу по именам сервисов (которые располагаются на следующем уровне после `services:` и обозначают объявление начала конфигурации конкретного контейнера)**

Основа docker-compose.yml для стека мониторинга

Давайте создадим файл-основу для нашего стека:

```
version: '3'

services:

volumes:

networks:
  monitoring-stack:
    driver: bridge
    ipam:
      config:
        - subnet: 10.5.0.0/16
          gateway: 10.5.0.1
```

Добавляем Prometheus

А теперь опишем то, что у нас было в команде `docker run`:

```
version: '3'

services:
  prometheus:
    image: prom/prometheus:v2.47.2
    container_name: prometheus
    command: --web.enable-lifecycle --config.file=/etc/prometheus/prometheus.yml
    ports:
      - 9090:9090
    volumes:
      - ./prometheus:/etc/prometheus
      - prometheus-data:/prometheus
    networks:
      - monitoring-stack
    restart: always

volumes:
  prometheus-data:
```

Описание раздела Prometheus

- Мы используем **образ prom/prometheus** с версией v2.47.2
- Контейнер будет называться **prometheus**
- Во время обращения на IP-адрес Docker сервера на порт 9090 он будет прокинут внутрь контейнера Prometheus на порт 9090

Описание раздела Prometheus

- Локальная директория на Docker сервере **./prometheus** прокидывается в **контейнер prometheus** в папку **/etc/prometheus**
- Создан Volume для сохранения данных директории **/prometheus контейнера**
- Контейнер использует сеть **monitoring-stack**
- Также контейнер запускается с параметром постоянного перезапуска в случае падения

Пробный запуск сценария

Теперь у нас есть конфигурация, включающая в себя Prometheus. Попробуем ее запустить:

```
sudo docker-compose up # запуск с выводом в консоль
```

```
sudo docker-compose up -d # запуск в бэкграунде
```

```
sudo docker-compose up -f anything.yml # запуск в случае если файл называется нетипично
```

Добавляем Pushgateway

Теперь давайте развернем средство для передачи метрик в Prometheus:

```
version: '3'

services:
  pushgateway:
    image: prom/pushgateway:v1.6.2
    container_name: pushgateway
    ports:
      - 9091:9091
    networks:
      - monitoring-stack
    depends_on:
      - prometheus
    restart: unless-stopped
```


Описание Pushgateway

- Мы используем образ **prom/pushgateway** с тегом (версией) [v1.6.2](#)
- Контейнер будет называться **pushgateway**
- Во время обращения на IP-адрес Docker сервера на порт **9091** он будет прокинут внутрь контейнера **pushgateway** на порт **9091**
- Контейнер использует сеть **monitoring-stack**
- Приложение будет запущено только после запуска **prometheus**
- Также контейнер будет перезапускаться в случае падения, если только он не был остановлен намеренно

Проверка Pushgateway

Нацеливаем prometheus на pushgateway:

```
# prometheus.yml

scrape_configs:
  - job_name: 'pushgateway'
    honor_labels: true
    static_configs:
      - targets: ['pushgateway:9091']
```

pushgateway (1/1 up) [show less](#)

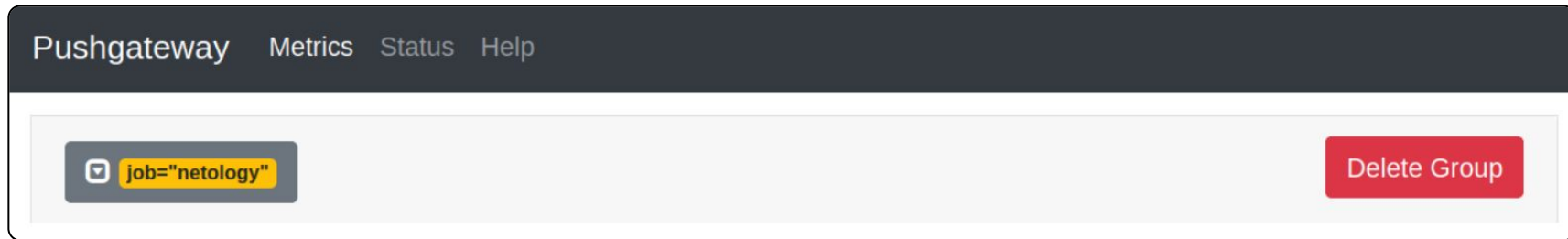
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://pushgateway:9091/metrics	UP	instance="pushgateway:9091" job="pushgateway"	14.606s ago	1.935ms	

Проверка Pushgateway

Пушим метрику в Pushgateway:

```
echo "docker 2" | curl --data-binary @-  
http://localhost:9091/metrics/job/netology
```

И наблюдаем ее в интерфейсе:



Добавляем Alertmanager

Теперь давайте развернем решение для оповещений о сбоях:

```
version: '3'

services:
  alertmanager:
    image: prom/alertmanager:v0.26.0
    container_name: alertmanager
    command: --config.file=/etc/alertmanager/alertmanager.yml
    ports:
      - 9093:9093
    volumes:
      - ./alertmanager:/etc/alertmanager
      - alertmanager-data:/data
    networks:
      - monitoring-stack
    depends_on:
      - prometheus
    restart: always

volumes:
  alertmanager-data:
```

Описание раздела Alertmanager

- Мы используем образ **prom/alertmanager** с конкретным тегом — версией **0.26.0**
- Контейнер будет называться **alertmanager**
- Во время обращения на IP-адрес Docker сервера на порт **9093** он будет прокинут внутрь контейнера **alertmanager** на порт **9093**

Описание раздела Alertmanager

- Локальная директория на Docker сервере **./alertmanager** прокидывается в контейнер **alertmanager** в папку **/etc/alertmanager**
- Создан Volume для сохранения данных директории **/data** контейнера
- Контейнер использует сеть **monitoring-stack**
- Приложение будет запущено только после запуска prometheus
- Также контейнер запускается с параметром постоянного перезапуска в случае падения

Проверка Alertmanager

Добавляем правило алертинга в конфигурацию prometheus и нацеливаем его на alertmanager:

```
# prometheus.yml

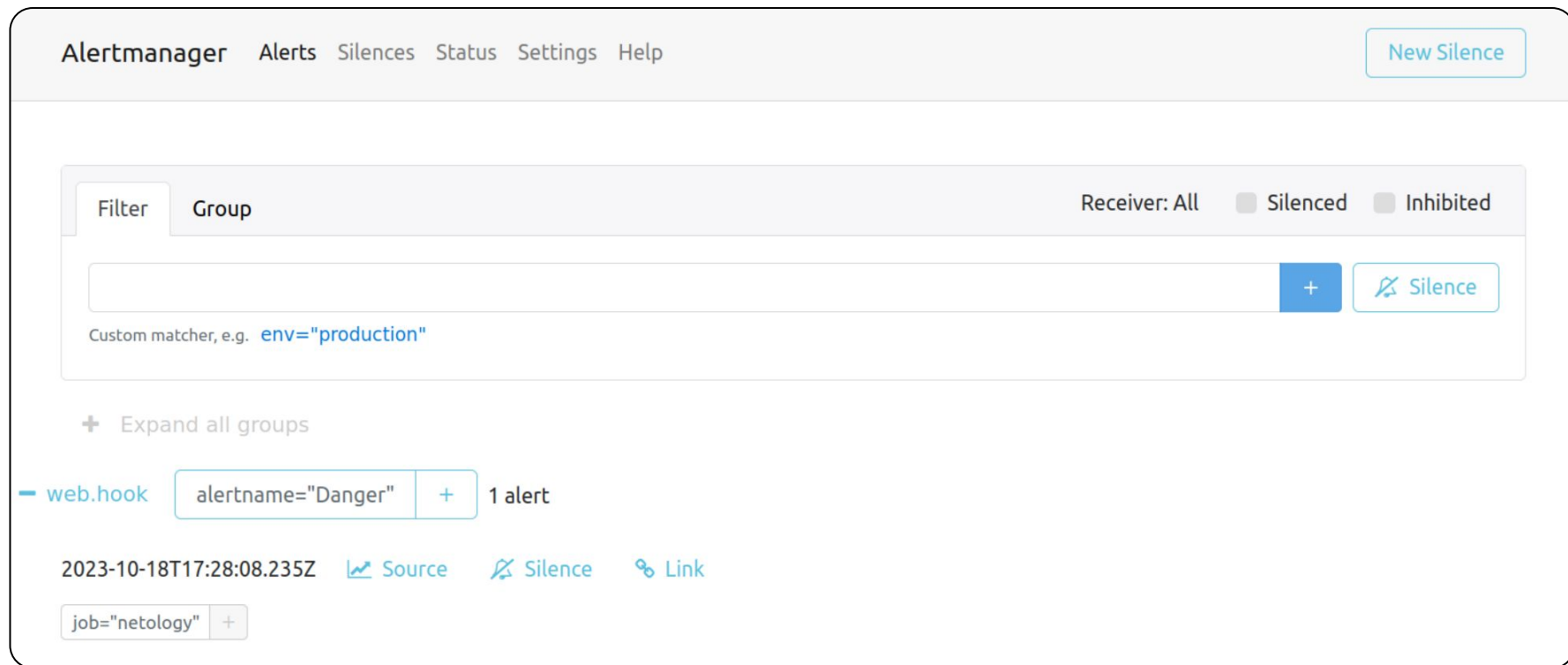
alerting:
  alertmanagers:
    - static_configs:
      - targets:
        - alertmanager:9093
rule_files:
  - alerts.yml

# alertls.yml

groups:
  - name: Netology
    rules:
      - alert: Danger
        expr: docker{job="netology"} > 1
        for: 10s
```

Проверка Alertmanager

Видим правило в интерфейсе alertmanager :



Добавляем сценарий для Grafana

Для визуализации полученных данных на графике добавим grafana:

```
version: '3'

services:
  grafana:
    image: grafana/grafana
    container_name: grafana
    environment:
      GF_PATHS_CONFIG: /etc/grafana/custom.ini
    ports:
      - 80:3000
    volumes:
      - ./grafana:/etc/grafana
      - grafana-data:/var/lib/grafana
    networks:
      - monitoring-stack
    depends_on:
      - prometheus
    restart: unless-stopped

volumes:
  grafana-data:
```

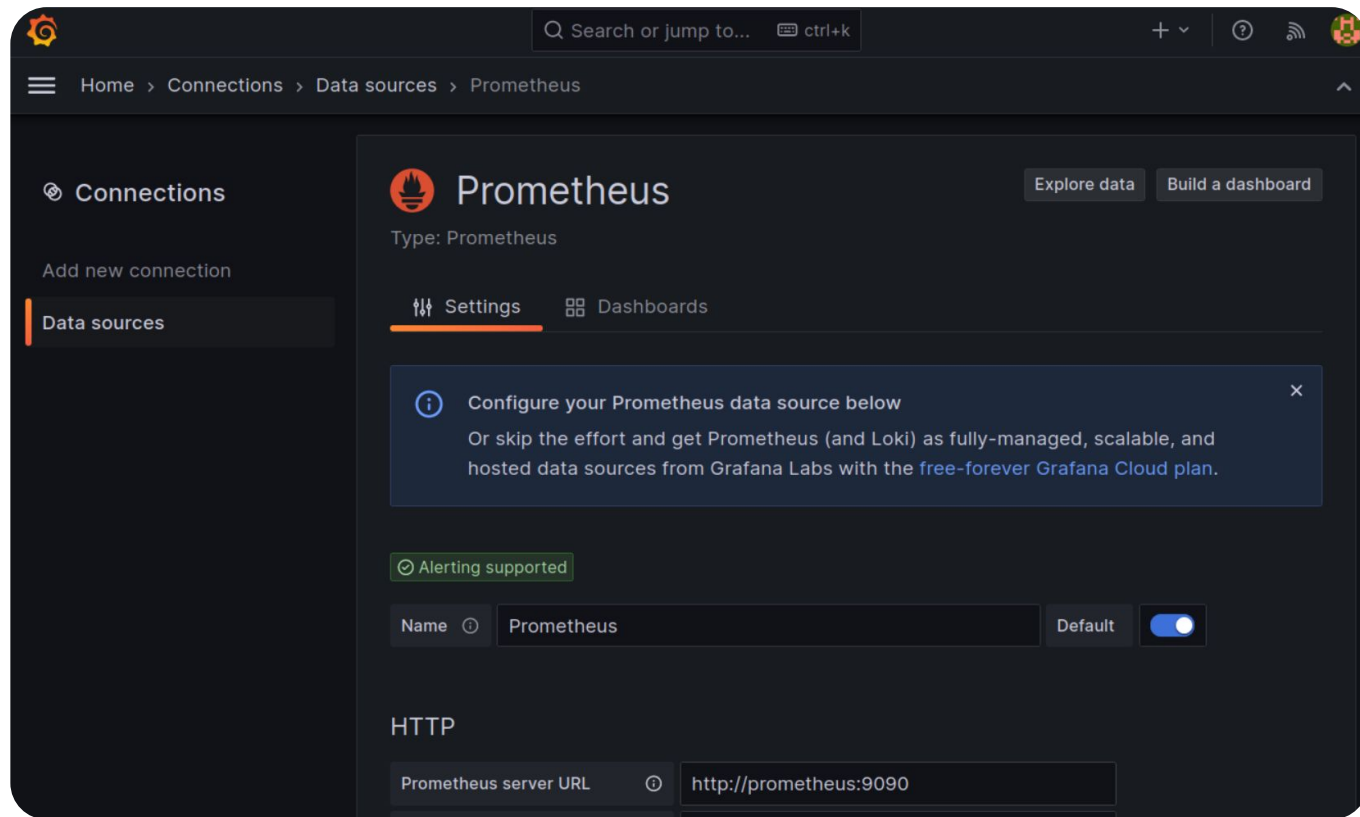
Описание раздела Grafana

- Мы используем образ **grafana/grafana** без тега, что автоматически означает тег **latest**
- Контейнер будет называться **grafana**
- С помощью переменной среды, поддерживаемой контейнером, мы задаём путь до кастомной конфигурации, где прописаны логин и пароль администратора
- Во время обращения на IP-адрес Docker сервера на порт **80** он будет прокинут внутрь контейнера **grafana** на порт **3000**

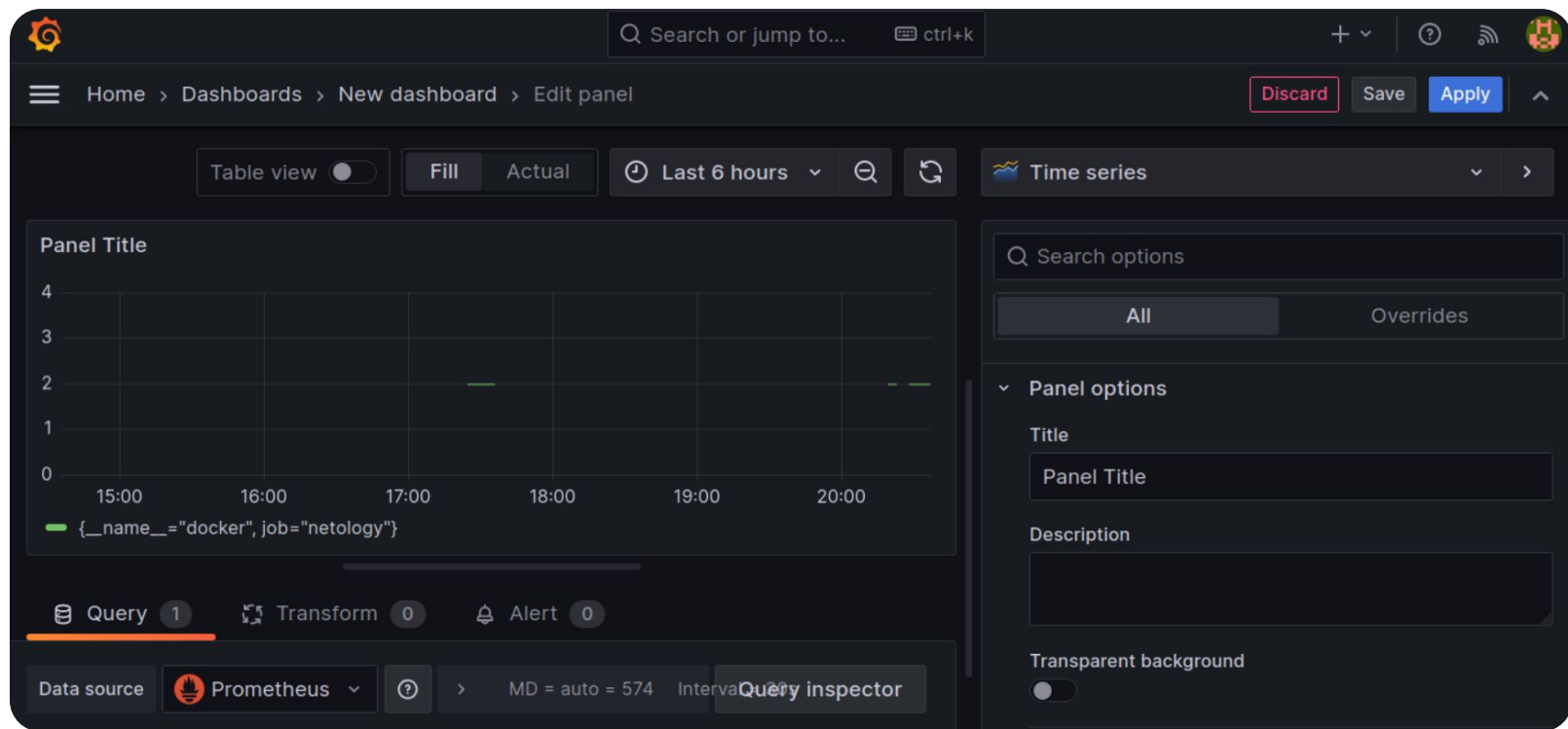
Описание раздела Grafana

- Локальная директория на Docker сервере **./grafana/provisioning** прокидывается в контейнер **grafana** в папку **/etc/grafana**
- Создан Volume для сохранения данных директории **/var/lib/grafana** контейнера
- Контейнер использует сеть **monitoring-stack**
- Приложение будет запущено только после запуска prometheus
- Также контейнер будет перезапускаться в случае падения, если он не был остановлен намеренно

Проверка Grafana: добавление Prometheus Data source



Проверка Grafana: построение графика на основе метрики из Pushgateway



Итоги занятия

Сегодня мы

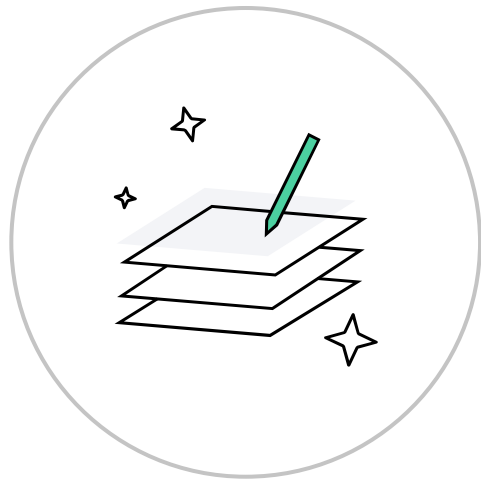
- Узнали, что такое Docker Compose и для чего он используется
- Разобрали, как работать с файлом docker-compose.yml
- Узнали как установить Docker Compose
- Запустили с помощью Docker Compose систему мониторинга на основе Prometheus из нескольких приложений



Домашнее задание

Давайте посмотрим ваше домашнее задание.

- 1 Вопросы по домашней работе задавайте в разделе "Вопросы по заданию"
- 2 Задачи можно сдавать по частям
- 3 Зачёт по домашней работе ставят после того, как приняты все задачи



Дополнительные материалы

- [Шпаргалка по docker compose](#)
- [Виды сетевых драйверов](#)
- [Про тома, их драйвера и режимы доступа](#)
- [Параметризация конфигурации](#)



**Задавайте вопросы
и пишите отзыв о лекции**

